AgentISA: Formal Instruction Set Design for LLM Agents

Anonymous Author(s)

Abstract

Large Language Models (LLMs) are increasingly deployed as autonomous agents, yet their behaviors are typically orchestrated through ad-hoc prompt engineering and per-provider function-calling interfaces. This makes agent behavior difficult to specify, analyze, and optimize systematically. We present AgentISA, an instruction set architecture for LLM-powered agents that treats agent behavior as a first-class compilable artifact. AgentISA contributes: (1) a formal instruction set with compositional semantics, a type system, and machine-checkable pre/post-conditions; (2) an Agentic Calling mechanism that lifts provider-specific function calling into reusable, cacheable agent instructions; (3) an automatic instruction learning system that mines recurring instruction patterns from interaction traces; and (4) a compiler and runtime applying classical optimization techniques—instruction fusion, dead-code elimination, pipeline scheduling, and dynamic hot-path specialization—to agent instruction streams.

We evaluate AgentISA with a reproducible microbenchmark suite that executes the real compiler, cache, and optimization-manager code paths against a mock LLM backend that counts simulated API calls and charges a fixed 80 ms simulated latency per call. On synthetic Zipf-distributed workloads (2,000 tasks, 5 random seeds), result caching reduces simulated API calls by 59.5%$±0.3±0.2±0.5±1.8±1.6±3.5±3.3±$4.6%. A scaled-down run against a live LLM gateway (200 tasks, one seed, one model; 1,040 real round trips) reproduces the exactly computed cache hit/miss sequence at every skew level and converts avoided calls into measured wall-clock and token savings, validating the caching mechanism end-to-end while remaining far short of production evidence. A suite of 43 multi-agent coordination and security tests provides functional validation of role assignment, capability-based agent selection, barrier synchronization, and authentication/authorization monitoring. We position these results honestly as microbenchmark evidence on synthetic workloads—not as production end-to-end measurements—and discuss the resulting threats to validity.

Keywords

- Instruction Set Architecture
- Large Language Models
- AI Agent Programming
- Compiler Optimization
- Multi-agent Systems
- Agentic Calling
- Automated Instruction Learning
- Cross-model Capability Assessment
- Formal Methods
- Performance Optimization

Research Highlights

1. A formal instruction set architecture that makes LLM agent behavior compilable, analyzable, and optimizable
2. Result caching reduces simulated API calls by 59.5–79.6% on synthetic Zipf workloads (5 seeds), with the reduction growing as workload skew increases
3. Static compilation (fusion + dead-code elimination) reduces the effective instruction count by 34.2%$±$1.8%, validated by a per-pass ablation
4. Dynamic hot-path optimization shrinks repeatedly executed traces by 65.8%$±$4.6%
5. Agentic Calling lifts provider-specific function calling into reusable, portable agent instructions
6. Against a live LLM gateway the cache reproduces the exactly predicted hit/miss sequence, converting avoided calls into measured wall-clock (14-46% of the no-cache arm) and token savings

CCS Concepts

- **Software and its engineering** → **Compilers**: instruction selection, optimization, and scheduling applied to agent instruction streams
- **Theory of computation** → **Program reasoning**: type safety and semantic-preservation proofs for instruction-level transformations
- **Computing methodologies** → **Artificial intelligence**: LLM-based agents, tool use, and multi-agent coordination
- **Computer systems organization** → **Architectures**: instruction set design and runtime support for agent execution

Funding Information

No external funding was received for this work.

1 Introduction

Recent advances in Large Language Models (LLMs) have produced systems with strong natural language understanding, generation, and—through instruction tuning and human feedback—instruction-following capabilities [1, 2, 3, 4, 5, 6]. Building on these capabilities, a rapidly growing body of work couples LLMs with tools, memory, and control loops to construct autonomous agents [9, 10, 11, 12]. Yet the engineering substrate beneath these agents remains largely ad hoc: behaviors are specified as prompts and provider-specific function-calling schemas, which are difficult to specify precisely, analyze formally, or optimize systematically.

1.1 Background and Motivation

Function calling—selecting an API from a declared tool set and emitting structured arguments—has become the dominant interface between LLMs and external systems [13, 14, 15]. Standardization efforts such as the Model Context Protocol [18] and evaluation harnesses such as the Berkeley Function Calling Leaderboard [17] are consolidating this interface layer. However, the unit of interaction

remains the individual API call: each call is dispatched, priced, and forgotten independently, even when an agent repeatedly executes structurally identical work.

This per-call statelessness stands in sharp contrast to how computing systems have historically handled repetitive execution. Classical compilers and hardware translate recurring patterns into cheaper forms—instruction fusion, dead-code elimination, caching, and trace-based dynamic optimization [33, 34]. Our motivating hypothesis is that these same techniques apply to LLM agent workloads: if agent behavior is expressed as an explicit instruction stream rather than as opaque prompts, then recurring patterns become visible to a compiler and can be optimized with well-understood, semantics-preserving transformations.

1.2 Challenges in Current LLM-based Agent Systems

Five structural challenges motivate AgentISA:

1. **Per-call statelessness of tool interfaces.** Mainstream function-calling and tool-use protocols treat each call as an isolated event [13, 14, 18]. The provider has no notion of a *program* being executed, so repeated identical work incurs repeated cost; caching, when present, is a bolt-on afterthought rather than a semantic property of the execution model.
2. **Experience that does not accumulate as executable structure.** Agent paradigms such as Reflexion [10] and Voyager [12] do accumulate experience, but as verbal reflections or skill libraries consumed by the model at inference time. The lessons are not reified as reusable, statically optimizable instructions that a compiler can fuse, schedule, or eliminate.
3. **Absence of a shared behavioral contract.** Each multi-agent framework defines its own abstractions for roles, messages, and workflows [19, 20, 21, 22, 23, 24]. There is no instruction-level contract that is independent of any single framework, which fragments tooling and makes cross-framework analysis difficult.
4. **Cost pressure at the wrong layer.** FrugalGPT [28] demonstrates that LLM inference cost is a first-order concern, and the serving-systems community has attacked this below the API boundary with paged attention, continuous batching, KV-cache-centric architectures, and attention kernels [38, 39, 40, 41]. What is missing is the complementary, above-the-API layer: reducing the *number and size* of calls an agent issues in the first place.
5. **Integration complexity across providers.** Portability across LLM providers is handled today primarily at the library level [26, 27]. Agent logic written against one provider's function-calling surface does not transfer cleanly to another, and capability differences between providers are hard to reason about systematically [17].

1.3 Research Objectives and Contributions

This paper introduces AgentISA, an instruction set architecture that addresses these challenges by making agent behavior a first-class compilable artifact. Our contributions are:

1. **A formal instruction set** for LLM agents with compositional semantics, a type system, and pre/post-conditions on every instruction (Section 3.1), together with proofs of Turing completeness (Theorem 3.1) and type safety (Theorem 3.2).
2. **Agentic Calling**, a mechanism that lifts provider-specific function calling into reusable,

cacheable, semantically specified agent instructions, with a model-adapter layer for cross-provider portability (Section 3.2).

3. **Automatic instruction learning**, which mines frequent instruction sequences from execution traces using frequency-based sequence mining and clustering, and promotes verified recurring patterns into first-class instructions (Section 3.3).
4. **A compiler and runtime** that apply instruction selection, fusion, dead-code elimination, pipeline scheduling, and dynamic hot-path specialization to agent instruction streams (Sections 3.4 and 4), with semantic-preservation arguments for each transformation (Theorems 3.4–3.6, proofs in Appendix C).
5. **An empirical microbenchmark evaluation** of the implemented system on synthetic Zipf-distributed workloads, reporting simulated API-call reduction from caching (59.5–79.6% depending on skew), effective instruction-count reduction from static compilation (34.2%$±1.8±$4.6%), each over 5 random seeds with full methodology disclosure (Section 5).
6. **Multi-agent coordination infrastructure** with capability-based agent selection, execution planning, fallback strategies, and a security monitor enforcing authentication, authorization, resource-access, and data-sensitivity policies (Section 3.5 and Section 5.6), validated by 43 passing functional tests.

We deliberately scope our empirical claims: all quantitative results come from microbenchmarks that exercise the real `agentflow.core.isa` implementation against a mock LLM backend on synthetic workloads. They establish that the optimizations are real and measurable in a controlled setting; they do not constitute production end-to-end measurements, and we discuss this limitation explicitly in Section 5.8.

The rest of this paper is organized as follows: Section 2 reviews related work. Section 3 details the design of AgentISA. Section 4 describes the implementation. Section 5 presents the empirical evaluation. Section 6 discusses limitations and future work, and Section 7 concludes.

2 Related Work

AgentISA draws on eight lines of work: foundation LLMs and their function-calling interfaces, reasoning-and-acting agent paradigms, tool-calling standardization, multi-agent frameworks, prompt compilation and cost optimization, agent benchmarks, classical compiler and architecture techniques, and LLM serving systems. We review each in turn and position our contributions.

2.1 Foundation LLMs and Function Calling

Modern LLMs are autoregressive transformers trained at large scale [1, 2, 3, 4], with instruction tuning and reinforcement learning from human feedback making them reliable instruction followers [5, 6]. Their text distribution is conventionally factorized as:

$$P(y|x) = \prod_{t=1}^{T} P(y_t|x, y_{<t}; \theta)$$

where x is the input context, y the generated output, and θ the model parameters. Function calling constrains this distribution to a structured output space:

$$F(x) = \text{argmax}_{f \in \mathcal{F}} P(f|x, \mathcal{F})$$

where $\mathcal{F}$ is the set of declared functions and $F(x)$ the selected call. GPT-4 [3], PaLM [2], and open models such as LLaMA [4] expose this capability with provider-specific schemas. AgentISA does not modify this layer; it treats any such interface as a *backend* to be targeted by a common instruction set, in the same way that a compiler targets multiple processor backends [33].

The cost structure of this layer motivates our work. If each call carries latency τ_i plus a context-switching overhead δ_i (prompt reconstruction, serialization, cache misses), then n similar calls cost:

$$O(n) = \sum_{i=1}^{n} (\tau_i + \delta_i)$$

Definition 2.1 (Function Call Overhead): The overhead of executing n function calls is the sum of per-call latencies and per-call context reconstruction costs, as above. When successive calls are structurally identical, δ_i is pure waste—which is precisely the waste AgentISA's caching and fusion are designed to remove.

2.2 Reasoning and Agent Paradigms

A second line of work enhances LLM reasoning at prompt time. Chain-of-thought prompting [7] and zero-shot reasoning [8] improve multi-step behavior by eliciting intermediate steps. ReAct interleaves reasoning traces with actions [9]; Reflexion adds verbal self-critique across episodes [10]; Tree of Thoughts explores deliberative search over thought trees [11]; and Voyager demonstrates open-ended skill acquisition in an embodied environment [12]. These paradigms define *what* an agent should do at the cognitive level. AgentISA is complementary: it defines *how* the resulting behaviors are represented, compiled, and executed as instructions. A ReAct loop, for instance, can be expressed as an AgentISA instruction sequence whose tool invocations become fused, cacheable agentic calls.

2.3 Tool Calling and Interface Standardization

Toolformer trains models to interleave tool calls with text [13]; Gorilla connects models to large API corpora [14]; ToolLLM scales tool coverage to 16,000+ real-world APIs [15]; and HuggingGPT orchestrates model hubs for task solving [16]. On the standardization side, the Model Context Protocol defines an open specification for exposing tools and context to LLM applications [18], and the Berkeley Function Calling Leaderboard provides a systematic cross-model evaluation of function-calling accuracy [17].

These efforts standardize the *interface*. AgentISA standardizes the *execution model above the interface*: it compiles a provider-independent instruction stream down to whatever interface a backend exposes. This distinction matters for portability—an AgentISA program compiled for two providers differs only in the final translation step (Section 3.2.3)—and for optimization, because interface-level standards alone cannot fuse or eliminate calls.

2.4 Multi-Agent Frameworks

AutoGen frames agent applications as multi-agent conversations [19]; MetaGPT assigns standardized roles and workflows to agents [20]; AgentVerse facilitates multi-agent collaboration and studies emergent behaviors [21]; ChatDev organizes agents as a software-development organization [22]; CAMEL studies communicative agent societies [23]; and AutoGPT popularized autonomous goal-driven agents [24].

AgentISA contributes a different artifact to this space: not a framework or orchestration philosophy, but an instruction-level substrate—capability profiles, coordination instructions with barrier semantics, and a security monitor—that any such framework could target. Concretely, our multi-agent layer (Section 3.5, Section 5.6) provides capability-based agent selection, execution planning with per-stage fallback strategies, and policy enforcement for authentication, authorization, resource access, and data sensitivity.

2.5 Prompt Compilation and Cost Optimization

Closest to our compilation perspective are systems that *compile* LLM invocations. DSPy compiles declarative prompt programs into optimized pipelines via prompt search and fine-tuning [25]. LangChain [26] and LlamaIndex [27] provide library-level abstractions for chaining LLM calls and indexing context. FrugalGPT reduces cost through model cascades—answering with cheaper models when possible [28].

AgentISA differs in the level at which optimization operates. DSPy optimizes *prompts*; FrugalGPT optimizes *model selection*; AgentISA optimizes *instruction streams*: instruction selection by dynamic programming over pattern covers, fusion, dead-code elimination, pipeline scheduling, result caching, and dynamic hot-path specialization (Sections 3.4 and 4). These are classical compiler transformations [33] applied to a new artifact, and they compose with—rather than replace—prompt-level and model-level optimization.

2.6 Benchmarks for Agent Systems

AgentBench evaluates LLMs as agents across diverse environments [29]; GAIA tests general assistant abilities [30]; PlanBench probes planning and reasoning about change [31]; and SWE-bench measures resolution of real-world GitHub issues [32]. These benchmarks evaluate *end-to-end capability*. AgentISA's evaluation instead measures *subsystem behavior under controlled conditions*—API-call reduction under synthetic Zipf workloads, instruction-count reduction from compilation, and trace shrinkage from dynamic optimization (Section 5). We view the two as complementary: instruction-level metrics explain *where* end-to-end gains come from, and are reproducible without live model access.

2.7 Classical Compiler and Architecture Techniques

AgentISA's optimization machinery is deliberately classical. Instruction selection by tree-pattern matching with dynamic programming, peephole optimization, and data-flow-based dead-code elimination are standard compiler techniques [33]. Pipelining, hazard detection, and resource-constrained scheduling follow the classical architecture literature [34]; the RISC-V project demonstrates the value of open, well-specified instruction sets [35]. Type safety via progress-and-preservation is the standard technique of programming-language theory [36], and CompCert establishes that compiler correctness itself can be formally verified [37]. Our contribution is

not a new compiler technique but the demonstration that this machinery transfers to LLM agent workloads, together with the semantic-preservation arguments adapted to instructions with pre/post-conditions (Appendix C).

2.8 LLM Serving Systems

Below the API boundary, the serving community has made inference dramatically cheaper: PagedAttention manages KV memory like virtual memory [38]; Orca introduces continuous batching for transformer serving [39]; FlashAttention redesigns attention for IO-awareness [40]; and Mooncake proposes a KVCache-centric disaggregated architecture [41]. Energy analyses quantify the inference cost of large models [42]. This line of work and AgentISA are orthogonal layers of the same stack: serving systems make each call cheaper; AgentISA reduces how many calls an agent makes and how much redundant work each carries. The two compound multiplicatively.

2.9 Research Gaps and Our Contributions

In summary, the literature leaves four gaps that AgentISA addresses:

1. **No instruction-level contract for agent behavior.** Interfaces are standardized [13, 14, 18], and frameworks are numerous [19, 20, 21, 22, 23, 24], but no shared instruction set spans providers and frameworks.
2. **No compiler for agent behavior.** Prompt-level compilation exists [25] and model-level cost reduction exists [28], but instruction-level fusion, elimination, scheduling, and caching do not.
3. **No semantics-preserving account of agent optimization.** Serving work verifies kernels and schedulers, not agent-level transformations; we provide progress-and-preservation-style arguments for ours (Theorems 3.2, 3.4, 3.5, 3.6).
4. **No reproducible instruction-level evaluation.** Existing benchmarks measure end-to-end capability [29, 30, 31, 32]; we contribute a seed-controlled microbenchmark suite with fully disclosed synthetic workloads and mock-backend instrumentation (Section 5).

Our work addresses these gaps through: (1) the Agentic Calling abstraction (Section 3.2), (2) pattern-based instruction optimization (Section 3.4), (3) automated instruction learning from execution traces (Section 3.3), and (4) a microbenchmark evaluation with explicit validity threats (Section 5).

3 AgentISA Design

Building upon the foundations established in Sections 1 and 2, we present AgentISA, a comprehensive instruction set architecture designed specifically for LLM-powered AI agents. This section details the core architecture, the Agentic Calling Framework, and the automatic instruction learning system.

3.1 Core Architecture

3.1.1 Instruction Set Specification

The AgentISA instruction set is formally defined as a tuple:

Definition 3.1 (AgentISA Core):

$$\text{AgentISA} = (I, T, \Sigma, \Delta, \Phi)$$

where:

- I is the set of instructions
- T is the type system
- Σ is the state space
- Δ is the transition function set
- Φ is the verification function set

Following the conventions of classical instruction set design [33, 34, 35], we define the basic instruction format:

```
INSTRUCTION = {
    opcode: OPCODE,
    args: ARGS[],
    precondition: PRED,
    postcondition: PRED,
    context: CONTEXT
}
```

The instruction set is categorized into four fundamental classes:

1. **Control Flow Instructions**:

$$\mathcal{C} = \{seq, branch, loop, call\}$$

2. **State Management Instructions**:

$$\mathcal{S} = \{load, store, update, cache\}$$

3. **LLM Interaction Instructions**:

$$\mathcal{L} = \{query, response, context\}$$

4. **Resource Management Instructions**:

$$\mathcal{R} = \{allocate, free, monitor\}$$

Theorem 3.1 (Instruction Completeness): The AgentISA instruction set is Turing complete.

Proof sketch: We exhibit a translation from WHILE programs—already known to be Turing equivalent—to AgentISA instruction sequences, using a counting argument over instruction classes: state management instructions encode an unbounded store, and control-flow instructions encode sequential composition, conditional branching, and iteration. The full construction and simulation argument are given in Appendix A, following standard simulation techniques from the compiler literature [33] and the operational-semantics framework of Pierce [36].

3.1.2 Type System and Semantics

The type system T uses dependent-type constructions in the style of Pierce [36]:

Definition 3.2 (AgentISA Type System):

$$T ::= \text{Base} \mid \Pi(x : T_1).T_2 \mid \Sigma(x : T_1).T_2 \mid \mu X.T$$

where:

- Base types include {Bool, Int, String, Float}
- Π represents dependent function types
- Σ represents dependent pair types
- μ represents recursive types

The semantic rules follow operational semantics:

$$\frac{e_1 \to e_1'}{E[e_1] \to E[e_1']} \text{ (Context)} \quad \frac{}{(\lambda x.e)v \to e[x := v]} \text{ (β-reduction)}$$

Type safety is guaranteed through the following theorem:

Theorem 3.2 (Type Safety): Well-typed AgentISA programs cannot "go wrong".

Proof: By the standard two-lemma technique—progress and preservation—as presented in Pierce [36]:

1. Progress: $\forall$e,T. $\vdash$e:T $\implies$ value(e) $\vee$ $\exists$e'. e$\to$e'
2. Preservation: $\forall$e,e',T. $\vdash$e:T $\wedge$ e$\to$e' $\implies$ $\vdash$e':T

Both lemmas are proved by simultaneous induction on typing derivations; the full proof, including the case analysis over instruction classes, is given in Appendix B.

3.1.3 LLM Integration Interface

The LLM integration interface is defined through a protocol, playing the same role for agent backends that the Model Context Protocol plays for tool exposure [18]:

$$\mathcal{P}_{LLM} = (Q, R, C, \psi)$$

where:

- Q: Query space
- R: Response space
- C: Context management functions
- ψ: State transformation functions

Following the interface conventions of modern function-calling providers [3, 18], we define the interface protocol:

```
Protocol LLMInterface {
    QueryFormat: JSON Schema
    ResponseFormat: JSON Schema
```

```
    ContextWindow: Integer
    StateMapping: Map<AgentState, LLMContext>
}
```

3.2 Agentic Calling Framework

3.2.1 Function Calling to Agentic Calling Transformation

The transformation from function calling to Agentic Calling is formalized as:

Definition 3.3 (Agentic Transformation):

$$\mathcal{T} : F \to A$$

where F is the function call space and A is the agent instruction space.

The transformation preserves semantic equivalence:

$$\forall f \in F, \exists a \in A : \text{sem}(f) \equiv \text{sem}(a)$$

The transformation process follows three steps, mirroring the signature-to-call conventions of function-calling interfaces [13, 14]:

1. Function signature analysis
2. Context embedding
3. Instruction synthesis

3.2.2 Context Preservation and State Management

State management in AgentISA follows a formal state machine model:

$$M = (S, s_0, \delta, F)$$

where:

- S: Set of states
- s_0: Initial state
- δ: State transition function
- F: Set of final states

Context preservation is guaranteed through:

Theorem 3.3 (Context Preservation): Let c be a context and t be a transformation, then:

$$\forall c \in C, \exists c' \in C' : \phi(t(c)) = \phi(c)$$

where φ is the context interpretation function.

Proof: The transformation t re-encodes the context into the instruction's `context` field without altering the semantic interpretation: by the definition of t, the extracted instruction carries a verbatim copy of the source context, and ϕ is defined compositionally over context contents, so $\phi(t(c)) = \phi(c)$ holds by construction. The adapter's `validateContext` check (Section 3.2.3) enforces this invariant at runtime. ■

3.2.3 Cross-model Compatibility

Cross-model compatibility is achieved through an abstract interface layer:

```
Interface ModelAdapter {
    translate: AgentInstruction → ModelSpecificCall
    normalize: ModelSpecificResponse → AgentResponse
    validateContext: Context → Boolean
}
```

Cross-model compatibility requirements follow the observation of cross-model function-calling studies [17] and open protocol efforts [18]: providers differ in surface syntax but can agree on semantics,

$$\forall m_1, m_2 \in M : \text{out}(m_1) \equiv \text{out}(m_2) \text{ when } \text{in}(m_1) \equiv \text{in}(m_2)$$

3.3 Automatic Instruction Learning

3.3.1 Dialogue Pattern Recognition

Pattern recognition in AgentISA is frequency-based and clustering-based rather than grammatical; it is implemented in `agentflow.core.isa.pattern_mining` and `agentflow.core.isa.instruction_learning`.

Definition 3.4 (Instruction Sequence Pattern): A pattern p over the instruction alphabet I is a non-empty contiguous sequence $p = (i_1, \ldots, i_k) \in I^k$ together with two frequency statistics:

- **Support**: $sup(p) = P(p \text{ occurs as a contiguous subsequence in a randomly drawn trace})$
- **Confidence**: $conf(p \Rightarrow j) = P(i_{t+1} = j \mid (i_{t-k+1}, \ldots, i_t) = p)$

A pattern is *frequent* if $sup(p) \geq \sigma_{min}$ and *predictive* if $\max_j conf(p \Rightarrow j) \geq \gamma_{min}$, for thresholds $\sigma_{min}, \gamma_{min} \in (0, 1]$.

Pattern recognition proceeds in three stages:

1. **Frequent-sequence mining.** Candidate subsequences are enumerated from execution traces together with their occurrence counts; sequences below σ_{min} are pruned, and successor statistics are accumulated for confidence estimation. The output is the set of frequent, predictive patterns.

2. **Feature extraction and clustering.** Each trace (or dialogue) is embedded into a feature vector—for instruction traces, instruction-type histograms, pattern occurrence counts, and dependency-graph statistics; for dialogues, TF-IDF vectors over significant turns. Vectors are normalized and clustered: k-means over instruction-trace features, and density-based clustering (DBSCAN) over dialogue embeddings. Clusters summarize distinct behavioral modes (e.g., retrieval-heavy versus reasoning-heavy traces) and parameterize per-cluster optimization decisions (Section 3.4.4).

3. **Pattern validation.** Each candidate pattern is checked for significance (frequency above threshold, non-trivial support, and—when derived from dialogues—an acceptable success rate among the dialogues exhibiting it) before it is promoted to a candidate instruction.

This design is deliberately statistical: agent traces are short and noisy, and frequency statistics degrade gracefully where grammar induction would overfit. It also composes with the compiler—a

frequent, verified pattern is exactly the candidate for promotion to a fused instruction (Section 3.4.4).

3.3.2 Instruction Extraction Pipeline

The instruction extraction process follows a sequential pipeline:

1. **Pattern Detection**:

$$P(pattern|dialogue) = \frac{P(dialogue|pattern)P(pattern)}{P(dialogue)}$$

2. **Instruction Formation**:

$$I = f(\text{pattern}, \text{context}, \text{constraints})$$

3. **Optimization**:

$$I_{opt} = \arg\min_{I' \in \mathcal{J}} \text{cost}(I')$$

3.3.3 Instruction Optimization and Verification

Optimization is coupled to verification, in the spirit of verified compilation [37] but at the granularity of individual instructions and transformations:

Definition 3.5 (Instruction Verification): An instruction i is valid if:

$$\forall s \in S : \text{pre}(i, s) \implies \text{post}(i, \text{exec}(i, s))$$

For instructions with finite, small state spaces, verification enumerates the reachable state space and checks the implication exhaustively; for larger state spaces, it checks a bounded set of symbolic states, which makes it a *bounded* verification procedure rather than a complete one. We state this honestly: Definition 3.5 defines validity, and our implementation approximates it by bounded exploration:

$$\forall s \in S_{bounded} \subseteq S : \text{pre}(i, s) \implies \text{post}(i, \text{exec}(i, s))$$

where $S_{bounded}$ is the explored subset. Completeness of the check is traded for tractability, following the standard operational-semantics treatment of instruction meaning [36].

Theorem 3.4 (Optimization Soundness): Optimized instructions preserve semantic equivalence.

Proof sketch: By structural induction on instruction composition. The base case shows each atomic rewrite (fusion, elimination, combining) preserves the denotation of a single instruction; the inductive step lifts preservation through each composition operator of Definition 3.6. The full proof, including the per-rewrite case analysis, is given in Appendix C.

Implementation details:

```
def verify_instruction(instruction, pre_condition, post_condition):
    state_space = generate_state_space(instruction)
    for state in state_space:
        if pre_condition(state):
            next_state = execute(instruction, state)
            assert post_condition(next_state)
```

3.4 Instruction Organization and Optimization

3.4.1 Instruction Composition

The AgentISA supports hierarchical instruction composition through a formal composition algebra:

Definition 3.6 (Instruction Composition): Given instructions i_1, $\mathrm{i}_2 \in \mathrm{I}$, we define composition operators:

$$\begin{aligned}&\text{Sequential} : i_1 \circ i_2\\ &\text{Parallel} : i_1 \parallel i_2\\ &\text{Conditional} : i_1 \rhd i_2\\ &\text{Iterative} : i_1^*\end{aligned}$$

Composition follows algebraic properties. Properties 1 and 3 hold for all instructions; Property 2 requires a side-effect-freeness restriction:

1. **Associativity** (all instructions, up to denotation):
$$(i_1 \circ i_2) \circ i_3 = i_1 \circ (i_2 \circ i_3)$$
2. **Distributivity** (only for side-effect-free instructions): Let i_1, i_2, i_3 be side-effect-free (pure) instructions, each denoting a state transformation that is independent of external state. For pure instructions, denotations are idempotent (applying the same update twice is equivalent to applying it once) and pairwise commutative (independent updates combine in any order), so both sides of
$$i_1 \parallel (i_2 \circ i_3) = (i_1 \parallel i_2) \circ (i_1 \parallel i_3)$$
denote the same net state transformation. Formally, the pure denotations form an idempotent, commutative structure under update combination, and $\parallel$ distributes over $\circ$ within it.
3. **Identity**:
$$i \circ \epsilon = \epsilon \circ i = i$$

where ϵ is the empty (identity) instruction.

Remark (necessity of the restriction): Distributivity fails for instructions with side effects. Consider i_1 a state-mutating instruction and $i_2 = i_3 = \epsilon$. The left side executes i_1 once; the right side executes i_1 twice ($i_1 \parallel i_2$ and again in $i_1 \parallel i_3$), so the two programs differ in the resulting state. The fusion and elimination rewrites of Section 3.4.4 therefore apply distributivity-based simplifications only after a purity check on the affected instructions; instructions declared `side_effects=True` are excluded from such rewrites.

3.4.2 Instruction Pipeline Design

We formalize the instruction pipeline as a directed acyclic graph (DAG):

Definition 3.7 (Pipeline DAG):

$$P = (V, E, \lambda, \delta)$$

where:

- V: Pipeline stages
- E: Dependencies between stages
- λ: Stage latency function
- δ: Data transfer function

Pipeline stages are organized as:

```
Pipeline {
    FETCH → DECODE → SCHEDULE → EXECUTE → COMMIT
    with
        hazard_detection: HazardGraph
        dependency_tracking: DependencyMatrix
        resource_allocation: ResourceMap
}
```

Theorem 3.5 (Pipeline Correctness): For any valid instruction sequence s, pipeline execution preserves program semantics.

Proof: By induction on pipeline stages and hazard resolution (details in Appendix C).

3.4.3 Instruction Selection and Scheduling

Instruction selection follows a tree-pattern matching approach:

Definition 3.8 (Selection Pattern):

$$\pi = (T, C, \text{cost}, \text{code})$$

where:

- T: Tree pattern
- C: Context constraints
- cost: Cost function
- code: Generated code

The selection process is formalized as:

$$\text{Select}(t) = \arg\min_{\pi \in \Pi} \{\text{cost}(\pi) \mid \text{matches}(\pi.T, t) \land \text{satisfies}(t, \pi.C)\}$$

Scheduling optimization problem:

$$\text{minimize} \sum_{i \in I} \text{delay}(i)$$

$$\text{subject to:}$$

$$\forall (i_1, i_2) \in \text{Deps} : \text{start}(i_2) \geq \text{end}(i_1)$$

$$\forall t : \sum_{i \in \text{active}(t)} \text{res}(i) \leq \text{Capacity}$$

3.4.4 Static and Dynamic Optimization

3.4.4.1 Static Optimization Static optimization employs several techniques:

1. **Peephole Optimization**:

$$\text{PH}(i_1, ..., i_n) = \begin{cases} \text{opt}(i_1, ..., i_n) & \text{if pattern exists} \\ i_1, ..., i_n & \text{otherwise} \end{cases}$$

2. **Dead Code Elimination**:

$$\text{Live}(i) = \text{Use}(i) \cup (\text{Live}_{\text{out}}(i) - \text{Def}(i))$$

3. **Instruction Combining**:

```
Combine Rules {
  LOAD followed by STORE $\rightarrow$ MOVE
  Multiple CONTEXT_UPDATE $\rightarrow$ Single UPDATE
  Sequential QUERY $\rightarrow$ Batch QUERY
}
```

3.4.4.2 Dynamic Optimization Dynamic optimization includes:

1. **Hot Path Detection**:

$$\text{Hot}(p) = \text{freq}(p) > \theta \wedge \text{stability}(p) > \alpha$$

2. **Trace Formation**:

$$\text{Trace}(i_1, ..., i_n) = \{(i_k, ..., i_{k+m}) \mid \text{freq}(i_k, ..., i_{k+m}) > \beta\}$$

3. **Adaptive Recompilation**:

```
def should_recompile(trace: Trace) -> bool:
    return (trace.execution_count > threshold and
            trace.performance < expected_performance)
```

3.4.5 Verification and Analysis

Optimization correctness is ensured through:

Theorem 3.6 (Optimization Soundness): For any optimization transform T,

$$\forall p \in \text{Program} : \text{sem}(T(p)) \equiv \text{sem}(p)$$

Proof Sketch:

1. Show semantic preservation for each atomic transformation (Appendix C, Lemma C.1)
2. Prove composition of transformations preserves semantics (Appendix C, Lemma C.2)
3. Demonstrate hazard-free execution in pipeline (Appendix C, Theorem 3.5)

Performance Model:

$$\text{Perf}(p) = \sum_{i \in p} (\text{base_cost}(i) + \text{overhead}(i) - \text{optimization_gain}(i))$$

where optimization_gain is calculated as:

$$\text{optimization_gain}(i) = \sum_{o \in \text{Opts}} \text{benefit}(o, i) \times \text{application_rate}(o, i)$$

3.5 Capability Assessment

Agents are heterogeneous: an agent (or the model behind it) may be strong at reasoning but weak at tool use, or vice versa. To make such differences explicit—and to give the multi-agent coordinator of Section 5.6 a principled basis for agent selection—AgentISA includes a capability assessment subsystem, implemented as `CapabilityProfile` in `agentflow.core.isa.capability`.

Definition 3.9 (Capability Profile): A capability profile of an agent a is a mapping from capability domains D (e.g., reasoning, tool use, communication, memory) to domain scores:

$$\text{Cap}(a) = \{(\text{dom}_k, s_k, w_k) \mid \text{dom}_k \in D\}$$

where each domain carries a score $s_k \in [0, 1]$ and a weight $w_k \geq 0$ reflecting the domain's importance in the current deployment context.

Assessment proceeds in three steps:

1. **Domain metric computation.** For each domain, `assess_capabilities` computes a set of `CapabilityMetric` values from the agent's instruction set—e.g., for the reasoning domain, metrics derived from the prevalence and complexity of reasoning-type instructions.
2. **Domain aggregation.** Metrics are aggregated into a per-domain `DomainScore`:

$$s_k = \frac{\sum_{m \in M_k} w_m \cdot \text{norm}(m)}{\sum_{m \in M_k} w_m}$$

where M_k is the metric set of domain k, w_m are metric weights, and $\text{norm}(\cdot)$ maps each metric to $[0, 1]$.

3. **Overall score.** `get_overall_score` computes the weighted mean across assessed domains:

$$S(a) = \frac{\sum_k w_k \, s_k}{\sum_k w_k}$$

falling back to the unweighted mean when all weights are zero.

Three properties of this design deserve emphasis. First, it is **static and instruction-derived**: scores are computed from declared instruction sets, not from live probes, so assessment is cheap, deterministic, and side-effect-free. Second, it is **compositional**: because scores summarize instructions, fusing or eliminating instructions during compilation (Section 3.4) changes the profile predictably, and the coordinator re-assesses after optimization. Third, it directly serves coordination: the agent-selection predicate in Section 5.6 chooses the highest-scoring agent above a capability threshold for each task, subject to the security policies of the `SecurityMonitor`.

We state the scope of this subsystem honestly: `CapabilityProfile` assesses *declared instruction-level capability*, which correlates with—but does not measure—end-to-end task performance. Cross-model behavioral comparison at benchmark level is the domain of evaluation suites such as BFCL [17] and AgentBench [29]; the two views are complementary.

4 Implementation

This section presents the implementation and validation of the AgentISA framework, with particular emphasis on the compiler infrastructure and optimization techniques described in Section 3.4.

4.1 Compiler Infrastructure

4.1.1 Instruction Selection Implementation

We implement the tree-pattern matching approach of Section 3.4.3 (Definition 3.8) using dynamic programming over the IR tree, in the classical manner of bottom-up instruction selection [33]:

```
Input: IR tree T, Pattern set P
Output: Optimal pattern selection S

function MATCH-TREE(T, P)
    Let costs[n] = ∞ for all nodes n in T
    Let selected[n] = ∅ for all nodes n in T

    function MATCH-SUBTREE(node)
        if costs[node] ≠ ∞ then
            return costs[node]
        end if

        min_cost ← ∞
        best_pat ← null

        for each pattern p in P do
            if MATCHES(p, node) then
                curr_cost ← PATTERN-COST(p)
```

```
                for each child c of node do
                    curr_cost ← curr_cost + MATCH-SUBTREE(c)
                end for

                if curr_cost < min_cost then
                    min_cost ← curr_cost
                    best_pat ← p
                end if
            end if
        end for

        costs[node] ← min_cost
        selected[node] ← best_pat
        return min_cost
    end function

    MATCH-SUBTREE(root(T))
    return (costs, selected)
end function
```

We validate the optimality of instruction selection through:

Theorem 4.1 (Selection Optimality): Given a set of patterns P and an IR tree T, the algorithm finds the minimal cost covering.

Proof: By induction on tree height:

1. Base case: Leaf nodes select optimal patterns directly
2. Inductive step: For a node n with children $c_1 \ldots c_n$:

$$\text{cost}(n) = \min_{p \in P}\{p.\text{cost} + \sum_{i=1}^{n} \text{cost}(c_i)\}$$

4.1.2 Pipeline Scheduling Implementation

We implement the pipeline scheduler following the DAG model from Section 3.4.2, with hazard detection and resource-constrained issue in the style of classical pipeline scheduling [34]:

```
Input: Instruction sequence I
Output: Schedule S with cycle assignments

function SCHEDULE-PIPELINE(I)
    D ← BUILD-DEPENDENCY-GRAPH(I)
    S ← empty schedule
    R ← priority queue of ready instructions

    while D not empty do
        c ← CURRENT-CYCLE()

        // Find newly ready instructions
        N ← GET-READY-NODES(D, c)
```

```
        ADD-TO-QUEUE(R, N)

        // Schedule instructions for current cycle
        while R not empty and HAS-RESOURCES(c) do
            i ← POP-HIGHEST-PRIORITY(R)
            slot ← ALLOCATE-RESOURCES(i, c)
            ADD-TO-SCHEDULE(S, i, slot)
            UPDATE-DEPENDENCIES(D, i)
        end while
    end while

    VERIFY-SCHEDULE(S)
    return S
end function
```

Schedule verification follows the formal model:

$$\forall i_1, i_2 \in \text{Schedule} : \text{depends}(i_1, i_2) \implies \text{slot}(i_1) < \text{slot}(i_2)$$

4.1.3 Static Optimization Implementation

We implement the optimization passes defined in Section 3.4.4:

1. **Peephole Optimization**:

```python
class PeepholeOptimizer:
    def optimize(self, instructions: List[Instruction]) -> List[Instruction]:
        patterns = self.load_patterns()
        optimized = []
        i = 0
        while i < len(instructions):
            matched = False
            for pattern in patterns:
                if pattern.matches(instructions[i:i+pattern.length]):
                    optimized.extend(pattern.replace(instructions[i:i+pattern.length]))
                    i += pattern.length
                    matched = True
                    break
            if not matched:
                optimized.append(instructions[i])
                i += 1
        return optimized
```

Optimization correctness is verified through:

$$\begin{aligned}&\forall p \in \text{Patterns}, i \in \text{Instructions} : \\ &\text{matches}(p, i) \implies \text{sem}(\text{replace}(p, i)) \equiv \text{sem}(i)\end{aligned}$$

2. **Instruction Combining**:

```
def combine_instructions(self, insts: List[Instruction]) -> List[Instruction]:
    combined = []
    current_group = []

    for inst in insts:
        if self.can_combine(current_group, inst):
            current_group.append(inst)
        else:
            if current_group:
                combined.append(self.merge_group(current_group))
            current_group = [inst]

    return self.verify_combinations(combined)
```

4.2 Runtime System

4.2.1 Dynamic Pipeline Control

Implementation of the dynamic pipeline controller:

```
class PipelineController:
    def __init__(self):
        self.hazard_detector = HazardDetector()
        self.resource_monitor = ResourceMonitor()
        self.performance_tracker = PerformanceTracker()

    async def execute_pipeline(self,
                               instruction_stream: AsyncIterator[Instruction]
                              ) -> AsyncIterator[Result]:
        async for inst in instruction_stream:
            # Hazard detection
            while self.hazard_detector.has_hazard(inst):
                await self.hazard_detector.wait_resolution()

            # Resource allocation
            resources = await self.resource_monitor.allocate(inst)

            try:
                # Execution
                result = await self.execute_instruction(inst, resources)
                # Performance tracking
                self.performance_tracker.record(inst, result)
                yield result
            finally:
                self.resource_monitor.release(resources)
```

Pipeline performance is modeled as:

$$\text{Throughput} = \frac{1}{\max(\text{Stage}_i)} \text{ where Stage}_i = \sum_{j \in \text{active}(i)} \text{latency}(j)$$

4.2.2 Dynamic Optimization Implementation

1. **Hot Path Detection**:

```
Input: Execution path P, Threshold θ, Window size w
Output: Boolean indicating if P is hot

function DETECT-HOT-PATH(P, θ, w)
    Let counter be an exponential moving average with window w

    function UPDATE-COUNTERS(path)
        h ← COMPUTE-PATH-HASH(path)
        counter[h] ← counter[h] × (1 - α) + 1

        if IS-PATH-HOT(path) then
            TRIGGER-OPTIMIZATION(path)
        end if
    end function

    function IS-PATH-HOT(path)
        h ← COMPUTE-PATH-HASH(path)
        freq ← counter[h] / TOTAL-EXECUTIONS()
        stab ← COMPUTE-STABILITY(path)
        return freq > θ and stab > STABILITY-THRESHOLD
    end function

    UPDATE-COUNTERS(P)
    return IS-PATH-HOT(P)
end function
```

Hot path probability is calculated as:

$$P(\text{hot}|\text{path}) = \frac{\text{freq}(\text{path})}{\text{total_exec}} \cdot \text{stability}(\text{path})$$

2. **Trace Formation**:

```
Input: Hot path H
Output: Optimized trace T

function BUILD-TRACE(H)
    T ← empty trace

    function SHOULD-INCLUDE(block)
        return EXECUTION-FREQUENCY(block) > THRESHOLD and
               PREDICTABLE-BRANCHES(block)
```

```
    end function

    function SHOULD-SPLIT(block)
        return HAS-SIDE-EXIT(block) or
               EXECUTION-FREQUENCY(block) < MIN-FREQ
    end function

    for each block B in H do
        if SHOULD-INCLUDE(B) then
            APPEND-TO-TRACE(T, B)
        else if SHOULD-SPLIT(B) then
            break
        end if
    end for

    VERIFY-TRACE-PROPERTIES(T)
    return T
end function
```

Trace validity is verified through:

$$\text{Valid(trace)} \iff \begin{cases} \text{Entry(trace) is dominated by header} \\ \text{Exit(trace) post-dominates tail} \\ \text{NoSideExit(trace) except tail} \end{cases}$$

4.3 Optimization Verification

4.3.1 Static Analysis

We implement verification of the static optimizations from Section 3.4.4:

```
Input: Original CFG O, Transformed CFG T
Output: Boolean indicating semantic equivalence

function VERIFY-TRANSFORMATION(O, T)
    // Build symbolic execution paths
    P₁ ← SYMBOLIC-EXECUTE(O)
    P₂ ← SYMBOLIC-EXECUTE(T)

    function VERIFY-EQUIVALENCE(path₁, path₂)
        for each symbolic state σ do
            if EVALUATE(path₁, σ) ≠ EVALUATE(path₂, σ) then
                return false
            end if
        end for
        return true
    end function

    // Verify all path pairs
```

```
    for each (p₁, p₂) in ZIP(P₁, P₂) do
        if not VERIFY-EQUIVALENCE(p₁, p₂) then
            return false
        end if
    end for

    return true
end function
```

The verification follows the formal model:

$$\text{Equiv}(p_1, p_2) \iff \forall \sigma \in \Sigma : [\![p_1]\!]\sigma = [\![p_2]\!]\sigma$$

4.3.2 Dynamic Analysis

Runtime verification implements:

1. **Performance Monitoring**:

```
class PerformanceMonitor:
    def monitor_execution(self,
                          instruction: Instruction,
                          context: Context) -> Metrics:
        start = time.perf_counter_ns()
        result = self.execute(instruction, context)
        end = time.perf_counter_ns()

        return Metrics(
            latency=(end - start),
            resource_usage=self.measure_resources(),
            correctness=self.verify_result(result, instruction)
        )
```

2. **Weighted Improvement Metric**: Optimization effectiveness is summarized as a descriptive weighted mean of metric deltas:

$$\text{Improvement} = \frac{\sum_{i=1}^{n} w_i (m_i^{\text{opt}} - m_i^{\text{base}})}{\sum_{i=1}^{n} w_i}$$

where m_i are performance metrics and w_i are importance weights. This is a descriptive summary only; the evaluation protocol of Section 5.1.3 reports mean $\pm$ standard deviation and makes no inferential (hypothesis-test) claims.

4.3.3 Empirical Results

We validate the implemented optimizations with the microbenchmark suite of Section 5, which drives the real compiler, cache, and optimization-manager code paths against a mock LLM backend. Summary of the results (all mean$\pm$stdev over 5 random seeds on synthetic Zipf workloads; full methodology and tables in Section 5):

Optimization	Measured effect	Where reported
Result caching	59.5%–79.6% reduction in simulated API calls, increasing with Zipf skew	Section 5.2
Static compilation (fusion + DCE)	34.2%$±1.8|*Dynamichot* − *pathoptimization*|65.8±$4.6% trace shrinkage on repeatedly executed traces	Section 5.4

These are microbenchmark measurements on synthetic workloads with simulated latency (80 ms per simulated call); they are not production end-to-end results. Section 5.7 reports a scaled-down validation of the caching mechanism against a real LLM gateway, and Section 5.8 discusses the remaining threats to validity.

5 Empirical Evaluation

We evaluate AgentISA with a reproducible microbenchmark suite (`examples/benchmarks/isa_microbench.py`) and a functional test suite for the multi-agent layer. We state the scope up front: the quantitative results in Sections 5.2-5.5 come from synthetic workloads executed against a *mock* LLM backend that counts simulated API calls and charges simulated latency; Section 5.7 then repeats the caching experiment against a live LLM gateway at reduced scale. They demonstrate that the optimizations are real, measurable, and reproducible in a controlled setting. They are **not** production end-to-end measurements, and no claim in this paper should be read as one. Section 5.7 validates the caching mechanism end-to-end against a real LLM gateway at reduced scale, and Section 5.8 discusses the threats to validity that follow.

5.1 Methodology

5.1.1 Instrumentation: Mock LLM Backend

All experiments route model inference through a `MockLLMBackend` that (1) counts each `infer` call as one simulated LLM API round trip, and (2) charges a fixed 80 ms simulated latency per call to a simulated clock. The backend never sleeps and never contacts a live model, which makes every "API call" observable, deterministic given a seed, and reproducible without network access or inference spend. Crucially, everything *around* the backend is real: the cache, compiler, and optimizer are the production `agentflow.core.isa` implementations (`InstructionCache`, `InstructionCompiler`, `OptimizationManager`), exercised through the same code paths a deployment would use.

5.1.2 Workloads

Workloads are synthetic and seed-controlled:

- **Caching**: 2,000 tasks drawn from a discrete distribution over 50 instruction templates × 20 parameter values, with task popularity following a Zipf distribution with exponent $s \in \{0.5, 1.0, 1.5\}$. Increasing s concentrates popularity on fewer tasks, increasing reuse potential.
- **Static compilation**: random instruction sequences of length 50, 200, and 1,000, compiled at optimization levels NONE, BASIC, ADVANCED, and AGGRESSIVE.
- **Dynamic optimization**: instruction traces of length 100 executed repeatedly (100 iterations), with hot-path detection active.
- **Ablation**: identical workloads with the fusion and dead-code-elimination passes enabled individually and jointly.

5.1.3 Protocol and Statistics

Each configuration is run with 5 random seeds; we report mean ± sample standard deviation across seeds. These are **descriptive statistics only**: we make no hypothesis-test claims (no p-values, no significance tests) anywhere in this paper. The seed count suffices to show that the effects are stable across workload instantiations—the standard deviations are small relative to the means—but a formal inferential comparison against other systems is out of scope, because there is no baseline system that shares the mock-backend instrumentation.

5.2 Experiment 1: Result Caching

Research question: How many simulated LLM API calls does AgentISA's result caching eliminate, as a function of workload skew?

We execute 2,000 tasks per seed, once with caching disabled and once with the `InstructionCache` enabled, and measure simulated API calls at the backend. The reduction ratio and cache hit rate coincide by construction: every avoided call corresponds to exactly one cache hit.

Zipf exponent s	Calls (no cache)	Calls (cached)	Reduction	Hit rate
0.5	2,000	810.4 ± 5.7	59.5% ± 0.3%	59.5% ± 0.3%
1.0	2,000	642.0 ± 3.8	67.9% ± 0.2%	67.9% ± 0.2%
1.5	2,000	408.0 ± 9.4	79.6% ± 0.5%	79.6% ± 0.5%

Interpretation. The reduction grows monotonically with skew, from 59.5% under nearly uniform popularity ($s = 0.5$) to 79.6% under heavy concentration ($s = 1.5$). This matches the cache-aware expectation from Section 2.5: the upper bound on savings is the probability mass of repeated tasks, and Zipf skew directly controls that mass. In simulated latency terms, the cache saves 95,168 ± 455 ms, 108,640 ± 305 ms, and 127,360 ± 750 ms of simulated time at $s = 0.5, 1.0, 1.5$, respectively (avoided calls × 80 ms).

Two caveats keep this honest. First, these are *simulated* calls against a mock backend: the reduction is exact for this workload family, but real deployments face cache-invalidation, prompt-context, and embedding-variability effects that the mock abstracts away. Second, the absolute number depends on the workload's reuse structure; workloads without repetition (e.g., each task unique) would see zero benefit. What the experiment establishes is the *mechanism and its sensitivity to skew*, not a universal constant.

5.3 Experiment 2: Static Compilation

Research question: How much does the compiler reduce instruction sequences, and what do the different optimization levels contribute?

We compile random instruction sequences at four optimization levels and measure two distinct quantities. We separate them explicitly because they are easy to conflate:

- **Effective instruction count**: the number of instructions that remain in the optimized program after fusion and dead-code elimination. This measures program size.

- **Dispatch count**: the number of independent dispatch operations the runtime issues. Parallel grouping can fold several independent instructions into one dispatch *without changing the effective instruction count*.

Sequence length	Level	Effective-instruction reduction	Dispatch count
50	NONE	0%	50.0 ± 0.0
50	BASIC	32.8% ± 4.1%	33.6 ± 2.1
50	ADVANCED	32.8% ± 4.1%	11.6 ± 2.9
50	AGGRESSIVE	32.8% ± 4.1%	11.6 ± 2.9
200	NONE	0%	200.0 ± 0.0
200	BASIC	34.2% ± 1.8%	131.6 ± 3.6
200	ADVANCED	34.2% ± 1.8%	44.4 ± 3.4
200	AGGRESSIVE	34.2% ± 1.8%	44.4 ± 3.4
1,000	NONE	0%	1,000.0 ± 0.0
1,000	BASIC	35.4% ± 1.6%	646.0 ± 16.4
1,000	ADVANCED	35.4% ± 1.6%	199.6 ± 21.4
1,000	AGGRESSIVE	35.4% ± 1.6%	199.6 ± 21.4

Interpretation. Three observations:

1. **Effective instruction count reduction is level-stable**: BASIC, ADVANCED, and AGGRESSIVE produce identical reductions (34.2% ± 1.8% at length 200). The fusion and DCE passes implemented at BASIC level already capture the available gains on these synthetic sequences; the higher levels add *scheduling* improvements, not additional elimination. We report this as-is rather than claiming monotonic gains across levels.

2. **Dispatch count drops further with parallel grouping**: at length 200, dispatch falls from 131.6 (BASIC) to 44.4 (ADVANCED) while effective instruction reduction is unchanged. This is the dispatch-folding effect described above: independent instructions grouped into shared dispatches. Readers should interpret the 44.4 figure as a scheduling reduction, not a further program-size reduction.

3. **Gains are stable across sequence lengths**: 32.8%, 34.2%, 35.4% at lengths 50, 200, 1,000 respectively, with the tightest variance at length 200 ($±$1.8%). Length does not materially change the opportunity structure of these synthetic sequences.

Note again that effective-instruction reduction is a *program-size* metric. It translates to simulated API-call savings only where eliminated or fused instructions carry model inference; our caching experiment measures the complementary reuse effect.

5.4 Experiment 3: Dynamic Hot-Path Optimization

Research question: Does the dynamic optimizer detect and specialize repeatedly executed traces?

We execute an instruction trace of length 100 for 100 iterations with hot-path detection active. Once a path is flagged hot (at iteration 50 in all seeds), the runtime forms an optimized trace and subsequent iterations execute the shrunken form.

Seed	Original length	Optimized length	Shrink ratio
0	100	30	70.0%

Seed	Original length	Optimized length	Shrink ratio
1	100	41	59.0%
2	100	30	70.0%
3	100	36	64.0%
4	100	34	66.0%

Across seeds, hot-path optimization shrinks the trace by **65.8% ± 4.6%**. The seed-to-seed spread (59.0%–70.0%) reflects the variability in which sub-sequences each synthetic trace makes hot; the effect is present in all five seeds. As with the static experiments, this measures trace-length reduction on synthetic traces—workload-controlled evidence that the detection-formation-reoptimization loop functions, not a wall-clock speedup claim.

5.5 Ablation: Per-pass Contribution

To attribute the static gains, we re-run the compiler with individual passes toggled:

Configuration	Instruction-count reduction
none	0%
fusion only	21.9% ± 3.5%
DCE only	24.1% ± 3.3%
fusion + DCE	35.0% ± 3.7%
all passes	35.0% ± 3.7%

Interpretation. Fusion alone removes 21.9% and DCE alone 24.1%; combined they remove 35.0%—less than the sum of their individual gains, because the passes partially overlap: sequences that are fusable are often also dead, so the second pass finds fewer targets after the first has run. The "all passes" row equals fusion+DCE on these workloads, consistent with Observation 1 in Section 5.3: no additional pass contributes measurable reduction here. We report the absence of additional gain rather than dressing the higher levels in unearned credit.

5.6 Multi-Agent Coordination: Functional Validation

The multi-agent layer—`AgentCoordinator`, `SecurityMonitor`, and the collaborative instruction set—is validated functionally by 43 tests, all passing. We present this as **qualitative functional validation**: the tests verify that the coordination behaviors exist and are correct, not that they are fast. No performance number should be inferred from this subsection.

The coordination flow implemented by `AgentCoordinator`:

```
Input: Agent set A, Task list τs
Output: Coordinated solutions S

function COORDINATE-AGENTS(A, τs)
    for each task T in τs do
        AUTHENTICATE(task issuer)              // SecurityMonitor
        AUTHORIZE(task, role policy)           // SecurityMonitor
        ag ← SELECT-BY-CAPABILITY(A, T)        // CapabilityProfile (Section 3.5)
        plan ← BUILD-EXECUTION-PLAN(T)         // stages + coordination rules
```

```
        for each stage in plan do
            if stage fails and fallback(stage) exists then
                EXECUTE-FALLBACK(stage)
            else
                EXECUTE-STAGE(stage, ag)
            end if
        end for
        RECORD-HISTORY(T, outcome)
    end for
end function
```

What the 43 tests verify, grouped by subsystem:

1. **Agent registration and role policies** (registration creates policies; permissions differ by role; security requirements differ by role; trust levels are bounded and reward declared specializations).
2. **Task coordination** (successful coordination produces a solution; failed tasks are recorded in history; missing authentication and missing authorization raise; absence of suitable agents raises; agent selection respects a capability threshold; the selected coordinator is the highest-scoring agent).
3. **Execution planning** (plans have stage structure with coordination rules; every stage has fallback keys; fallback strategies are differentiated; a fallback actually executes on stage failure; validators are reasoning-heavy non-coordinator agents).
4. **SecurityMonitor** (valid tasks pass; missing authentication fails; missing authorization fails; violations are recorded with timestamps; declared data sensitivity is enforced; sensitive categories infer a high sensitivity level; benign categories pass at medium; resource access is open by default and restrictable by configuration).
5. **Collaborative instructions** (barrier synchronization waits for the declared number of participants and times out when a peer is missing; width-one barriers resolve immediately; resource-allocation instructions reject over-allocation and serialize concurrent requests; the workflow state machine accepts valid actions, rejects invalid ones and double starts; a multi-agent research-team scenario runs end to end).

These tests exercise the same production modules used elsewhere in the system and are fully deterministic (no LLM calls). They establish correctness of the coordination machinery under the tested conditions. What they do not establish is scalability under contention, crash recovery, or adversarial inputs beyond the policy checks above—all are future work (Section 6).

5.7 Real-LLM End-to-End Validation of Result Caching

Research question: does the caching mechanism behave identically when the "model" is a real LLM behind a live gateway, and how much *measured* wall-clock time and token usage does it save?

Setup. We replace the mock backend with a real model served by a local Anthropic-protocol gateway (TLS with an internal CA; alias `claude-glm-5.3-flash`, a hosted reasoning-capable GLM model; `max_tokens=64`). Requests are sequential HTTP calls (httpx, no proxy, 90 s timeout), with three bounded retries for transient failures — none occurred. Every other component is the production code path: the same `InstructionCache` protocol, the same instruction objects, the same workload generator and the same instruction pool (50 templates) and parameter space (20 values) as Section 5.2.

Scale. Because each real round trip costs ~2.2 s, we reduce scale from 2,000 tasks x 5 seeds to **200 tasks x 1 seed** per Zipf level (s in {0.5, 1.0, 1.5}); the whole run is 1,040 real round trips in 39.4 minutes. The seed pins the task sequence exactly, so the workload is the same one the expectation below is computed from.

An exactness property. A cache hit depends only on the (instruction, params) pair seen so far — not on what the model returns. The hit/miss sequence for a fixed workload is therefore *computable without any backend*. If the mechanism is correct, the real run must reproduce that sequence exactly. It does:

Zipf s	Expected hit rate (computed, no backend)	Observed hit rate	Calls (no cache -> cached)	Call reduction	Wall clock (no cache -> cached, ms)	Wall saved (ms)	Tokens saved
0.5	8.5%	8.5%	200 -> 183	8.5%	472,276 -> 405,754	66,522	1,445
1.0	26.0%	26.0%	200 -> 148	26.0%	455,946 -> 334,597	121,349	4,420
1.5	45.5%	45.5%	200 -> 109	45.5%	450,330 -> 244,316	206,014	7,735

Per-call latency was stable across arms (p50 2.13-2.18 s, p95 3.06-3.54 s). Token savings are gateway-reported `usage` differences: every avoided call saves its input tokens plus the tokens the reasoning model generates before responding (output tokens were the binding constraint of `max_tokens=64` in every call).

Interpretation. Three points. First, the mechanism transfers unchanged: observed hit rates match the computed expectation exactly at every skew level, so the synthetic results of Section 5.2 measure the cache, not an artifact of the mock. Second, avoided calls convert into measured time and tokens: 14.1%, 26.6%, and 45.8% of the no-cache arm's wall clock at s = 0.5, 1.0, 1.5. These measured savings track the call reduction but are not identical to it, because arms were measured sequentially and per-call latency drifted slightly between arms (the exact quantity is the call reduction; wall and token figures are measurements, not derivations). Third, the *absolute* hit rates here (8.5-45.5%) are lower than Section 5.2's (59.5-79.6%) purely because repetition accumulates with task count: at N = 200 these are the exact computed values for this workload family, and at N = 2,000 the same computation yields the Section 5.2 numbers. The dependence on N is a property of the workload, not the backend.

Scope of the claim. This experiment validates the caching mechanism end-to-end: one gateway, one model, one seed, N = 200, sequential requests, no streaming. The compiler and dynamic-optimization experiments (Sections 5.3-5.5) are structural — they do not dispatch to the backend at all — so the real-LLM run does not re-execute them; their validity rests on the code-path and ablation arguments made there. Nothing in this section is a throughput, availability, or dollar-cost claim.

5.8 Limitations and Threats to Validity

We enumerate the limitations of the evaluation explicitly:

1. **Mock backend, not a live LLM (partially addressed).** The mock charges a constant 80 ms per simulated call and returns deterministic responses. Section 5.7 addresses the *mechanism-transfer* part of this threat at reduced scale: against a live gateway, the cache reproduces the exactly computed hit/miss sequence, and avoided calls convert to measured

time and tokens. What remains untested is caching under real model nondeterminism *across prompt variations*, streaming responses, and provider rate limits.

2. **Synthetic workloads.** Zipf-distributed synthetic tasks are a reasonable model of skewed agent workloads, but they are a model. The cache-reduction figures (59.5–79.6%) are exact for these workload families and should be read as *skew-sensitivity curves*, not as universal savings. A deployment with uniformly novel tasks would see near-zero cache benefit.

3. **No production end-to-end evaluation.** We have not deployed AgentISA in a production setting. Consequently this paper contains no latency-under-load, throughput, availability, or dollar-cost measurements, and we make no such claims.

4. **No real cost data (token usage now measured; pricing still not modeled).** All latency figures in Sections 5.2-5.5 are simulated (avoided calls $\times$ 80 ms). Section 5.7 measures real wall-clock time and gateway-reported token usage at reduced scale, but no pricing is applied and no dollar-savings claim is made; a priced deployment study remains future work.

5. **Program-size metrics, not wall-clock time.** Effective-instruction reduction and dispatch-count reduction are structural metrics. They correlate with—but do not equal—wall-clock improvements, which depend on per-instruction cost, parallelism available in the runtime, and dispatch overhead.

6. **Descriptive statistics only.** Five seeds per configuration; mean $\pm$ sample stdev. No hypothesis tests, confidence intervals, or significance claims are made. The standard deviations indicate stability across seeds, nothing more.

7. **Functional, not performance, validation of coordination.** The 43 multi-agent tests verify behavioral correctness under deterministic conditions, not throughput or scalability.

Despite these limitations, the evaluation achieves its purpose: it demonstrates, under fully disclosed and reproducible conditions, that each optimization mechanism is implemented, active, and produces the claimed structural effects on workloads designed to expose them.

6 Discussion

6.1 System Limitations and Challenges

1. **Instruction Set Completeness**
   - Certain complex reasoning patterns are not yet naturally expressible; domain-specific instruction extensions are likely needed
   - Maintaining consistency of pre/post-conditions and typing rules across extensions remains an open engineering challenge
2. **Evaluation Scope**
   - As detailed in Section 5.8, our quantitative evidence is microbenchmark-based: synthetic Zipf workloads against a mock LLM backend
   - Production end-to-end evaluation—including real provider pricing, rate limits, and failure modes—is required before any deployment claim
3. **Performance Bottlenecks**
   - Cache memory consumption under large instruction universes
   - Synchronization overhead in barrier-coordinated multi-agent execution
   - Recompilation cost when hot-path thresholds are misconfigured

4. **Integration Challenges**
   - Adapter maintenance as provider function-calling surfaces evolve
   - Version management of learned instructions across LLM updates

6.2 Security and Privacy Considerations

AgentISA's implemented security layer is the `SecurityMonitor` of Section 5.6, which enforces task-level authentication, role-based authorization, data-sensitivity classification, and configurable resource-access policies; the coordinator consults it before every assignment, and violations are recorded with timestamps. Beyond what is implemented, we identify the following as design considerations for hardening the architecture—not as claims about the current system:

1. **Data privacy**: instruction-level access control exists via authorization policies; end-to-end data-flow tracking and privacy-preserving computation remain future work (Section 6.3.6).
2. **System security**: instruction pre/post-condition verification (Section 3.3.3) reduces misexecution risk; secure multi-agent communication channels and audit-log integrity are not yet addressed.
3. **Compliance**: deployments in regulated domains will require externally audited controls; the current system provides enforcement hooks, not certification.

6.3 Future Research Directions

6.3.1 Theoretical Foundations

1. **Formal Semantics of Agent Instructions**

   A complete formal semantic framework would consolidate the type system and operational semantics of Section 3.1:

   $$\mathcal{F} = (L, \Sigma, \mathcal{R}, \mathcal{T})$$

   where:

   - L: Agent instruction language
   - Σ: State space
   - $\mathcal{R}$: Reduction rules
   - $\mathcal{T}$: Type system

2. **Compositional Reasoning**

   Development of compositional proof systems for agent behaviors:

   $$\frac{\{P_1\}S_1\{Q_1\} \quad \{P_2\}S_2\{Q_2\}}{\{P_1 \wedge P_2\}S_1 \parallel S_2\{Q_1 \wedge Q_2\}}$$

   This enables formal verification of complex agent systems through:

   - Modular verification
   - Parallel composition rules
   - Temporal property preservation

3. **Type Theory for Agent Capabilities**

   A capability-based type system:

```
Type Capability = {
  Basic: {Perception, Action, Memory}
  Composite: Basic $\rightarrow$ Basic
  Emergent: P(Composite)
}
```

   With typing rules:

$$\frac{\Gamma \vdash a : \text{cap}_1 \quad \Gamma \vdash b : \text{cap}_2}{\Gamma \vdash a \circ b : \text{cap}_1 \otimes \text{cap}_2}$$

6.3.2 Advanced Instruction Set Evolution

1. **Self-Modifying Instructions**

   Formal model for instruction evolution:

$$I_{t+1} = f(I_t, \text{Performance}_t, \text{Context}_t)$$

   With constraints:

   - Semantic preservation
   - Performance monotonicity
   - Safety guarantees

2. **Context-Aware Optimization**

   Dynamic optimization framework:

   Algorithm 6: Context-Aware Optimization

```
Input: Instruction set I, Context history C
Output: Optimized instruction set I'

function OPTIMIZE-CONTEXT(I, C)
    Let M be a context-performance model
    Let R be the set of rewrite rules

    for each context c in C do
        P $\leftarrow$ PREDICT-PERFORMANCE(M, I, c)
        if P < THRESHOLD then
            candidates $\leftarrow$ GENERATE-VARIANTS(I, R)
            I' $\leftarrow$ SELECT-BEST(candidates, c)
            UPDATE-MODEL(M, I', c)
        end if
    end for

    return I'
end function
```

3. **Cross-Model Instruction Mapping**

   Universal mapping framework:

   $$\phi_{A \to B} : I_A \to I_B$$

   Properties:

   - Semantic preservation
   - Performance bounds
   - Capability awareness

6.3.3 Theoretical Performance Bounds

1. **Asymptotic Analysis**

   A natural next step is deriving cost bounds per composition operator—e.g., when does iterative composition i_1^* admit a closed-form cost model, and under what reuse structure does caching yield amortized $O(1)$ effective cost per task? Our microbenchmarks (Section 5) measure the constants empirically; the corresponding asymptotic theory remains open.

2. **Resource Consumption Models**

   A calibrated resource model:

   $$R(i) = \alpha C_{cpu}(i) + \beta C_{mem}(i) + \gamma C_{io}(i)$$

   where α, β, γ are platform-specific coefficients that should be fitted against real deployments rather than assumed.

6.3.4 Capability Assessment: Extensions

The instruction-level capability assessment subsystem (`CapabilityProfile`, Section 3.5) is implemented and used by the coordinator. Two extensions remain future work:

1. **Benchmark-calibrated metrics**: correlating static, instruction-derived domain scores with end-to-end benchmark performance, in the manner of function-calling evaluations [17] and agent benchmarks [29], so that weights w_k are empirically grounded rather than configured by hand.
2. **Cross-model comparison matrices**: extending the per-agent profile to a pairwise comparison framework $\Delta(M_1, M_2)$ over capability dimensions, supporting systematic model selection for a given instruction workload.

6.3.5 Emergent Behavior Analysis

1. **Collective Intelligence Modeling**

   Multi-agent emergence framework:

   $$E(A_1, ..., A_n) = \phi(\bigcup_{i=1}^{n} \text{Cap}(A_i)) - \sum_{i=1}^{n} \text{Cap}(A_i)$$

   where φ represents emergent properties.

2. **Learning Transfer Mechanisms**

   Knowledge transfer model:

   Algorithm 7: Knowledge Transfer

```
Input: Source agents S, Target agent T
Output: Enhanced target agent T'

function TRANSFER-KNOWLEDGE(S, T)
    K $\leftarrow$ EXTRACT-KNOWLEDGE(S)
    C $\leftarrow$ IDENTIFY-COMMON-PATTERNS(K)

    for each pattern p in C do
        if APPLICABLE(p, T) then
            T' $\leftarrow$ ADAPT-KNOWLEDGE(T, p)
            if VERIFY-TRANSFER(T') then
                UPDATE-AGENT(T, T')
            end if
        end if
    end for

    return T'
end function
```

6.3.6 Security and Privacy Guarantees

1. **Information Flow Control**

   Formal security model:

$$\text{Secure}(P) \iff \forall s_1, s_2 : L(s_1) = L(s_2) \implies L([\![P]\!]s_1) = L([\![P]\!]s_2)$$

   where L represents security levels.

2. **Privacy-Preserving Computation**

   Privacy guarantee framework:

$$\text{PrivacyLoss}(M) \leq \epsilon + \sum_{i=1}^{n} \delta_i$$

   where:

   - ε: Privacy budget
   - δ_i: Privacy loss per operation

These directions form a roadmap for moving AgentISA from microbenchmark-validated infrastructure to a deployable, formally verified agent platform. None of the frameworks above is implemented today; they are stated with enough precision to be falsifiable and to guide subsequent work.

7 Conclusion

This paper presented AgentISA, an instruction set architecture that treats LLM agent behavior as a first-class compilable artifact. Our contributions are:

1. A formal instruction set with compositional semantics, a type system, and pre/post-conditions, together with proofs of Turing completeness (Theorem 3.1), type safety (Theorem 3.2), and semantic preservation for the optimization and scheduling transformations (Theorems 3.4–3.6).
2. Agentic Calling, which lifts provider-specific function calling into reusable, portable, cacheable agent instructions.
3. Automatic instruction learning based on frequency mining and clustering of execution traces.
4. A compiler and runtime applying classical optimizations—fusion, dead-code elimination, scheduling, and dynamic hot-path specialization—to agent instruction streams.
5. A multi-agent coordination layer with capability-based selection and a security monitor enforcing authentication, authorization, and data-sensitivity policies.

The empirical evaluation is deliberately scoped and fully disclosed: a microbenchmark suite drives the real compiler, cache, and optimizer against a mock LLM backend on synthetic Zipf workloads, over 5 seeds per configuration. Under these conditions, result caching reduces simulated API calls by 59.5–79.6% (increasing with workload skew), static compilation reduces the effective instruction count by 34.2%$±1.8±3.5±3.3±$4.6%. The multi-agent layer is validated by 43 passing functional tests. We have been explicit throughout about what these results are not: the Section 5.2-5.5 figures are microbenchmark evidence on synthetic workloads, not production end-to-end measurements. The Section 5.7 run against a live gateway validates the caching mechanism end-to-end and measures real wall-clock and token savings at reduced scale, but dollar-cost, throughput, and availability claims remain out of scope.

AgentISA's longer-term significance is architectural: by giving agent behavior an instruction-level contract, it makes agent systems amenable to the same discipline that transformed hardware and compiler engineering—precise specification, portability across backends, and semantics-preserving optimization. Establishing the corresponding gains on production workloads, and hardening the formal guarantees into verified implementations, is the immediate next step.

Artifact Availability

All artifacts are available in the accompanying repository:

- the AgentISA implementation (`agentflow.core.isa`);
- the microbenchmark suite (`examples/benchmarks/isa_microbench.py`);
- the real-LLM benchmark (`examples/benchmarks/isa_real_llm_bench.py`);
- the raw results (`examples/benchmarks/isa_microbench_results.json` and `examples/benchmarks/isa_real_llm_results.json`);
- the 43 multi-agent functional tests (`tests/core/isa/test_coordination.py` and `tests/core/instructions/test_agent_coordination.py`).

The Section 5.2-5.5 numbers regenerate with:

```
./venv/bin/python examples/benchmarks/isa_microbench.py \
    --experiments cache compiler dynamic ablation --seeds 5
```

The Section 5.7 numbers regenerate against any Anthropic-protocol gateway (configure `ANTHROPIC_BASE_URL` and `ANTHROPIC_AUTH_TOKEN`; the paper's run used a local alias for a hosted GLM reasoning model) with:

```
./venv/bin/python examples/benchmarks/isa_real_llm_bench.py \
    --tasks 200 --seeds 1 --zipf 0.5 1.0 1.5 --max-tokens 64
```

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei. 2020. Language Models are Few-Shot Learners. NeurIPS 33. arXiv:2005.14165.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, Noah Fiedel. 2023. PaLM: Scaling Language Modeling with Pathways. JMLR 24(240):1–113. arXiv:2204.02311.

[3] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.

[5] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, Quoc V. Le. 2022. Finetuned Language Models Are Zero-Shot Learners. ICLR 2022 (Oral). arXiv:2109.01652.

[6] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, Ryan Lowe. 2022. Training language models to follow instructions with human feedback. NeurIPS 35. arXiv:2203.02155.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V. Le, Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 35, pp. 24824–24837. arXiv:2201.11903.

[8] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, Yusuke Iwasaka. 2022. Large Language Models are Zero-Shot Reasoners. NeurIPS 35. arXiv:2205.11916.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023 (notable top 5%). arXiv:2210.03629.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 36. arXiv:2303.11366.

[11] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 36. arXiv:2305.10601.

[12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

[13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 36. arXiv:2302.04761.

[14] Shishir G. Patil, Tianjun Zhang, Xin Wang, Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.

[15] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, Maosong Sun. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. ICLR 2024 (Spotlight). arXiv:2307.16789.

[16] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. NeurIPS 36. arXiv:2303.17580.

[17] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. ICML 2025.

[18] Anthropic. 2024. Introducing the Model Context Protocol. Anthropic announcement, November 25, 2024.

[19] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (Final version COLM 2024.)

[20] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024 (Oral). arXiv:2308.00352.

[21] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu,

Maosong Sun, Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. ICLR 2024 (Poster). arXiv:2308.10848.

[22] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. ACL 2024, pp. 15174–15186. arXiv:2307.07924.

[23] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. NeurIPS 36. arXiv:2303.17760.

[24] Significant Gravitas. 2023. AutoGPT. GitHub repository. https://github.com/Significant-Gravitas/AutoGPT

[25] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, Christopher Potts. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714.

[26] Chase, Harrison. 2022. LangChain. GitHub repository. https://github.com/langchain-ai/langchain

[27] Liu, Jerry. 2022. LlamaIndex. GitHub repository. https://github.com/run-llama/llama_index

[28] Lingjiao Chen, Matei Zaharia, James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

[29] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. ICLR 2024. arXiv:2308.03688.

[30] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983. (Final version ICLR 2024.)

[31] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, Subbarao Kambhampati. 2023. PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. NeurIPS 2023 Datasets and Benchmarks Track. arXiv:2206.10498.

[32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? ICLR 2024 (Oral). arXiv:2310.06770.

[33] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. 2006. Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.

[34] John L. Hennessy, David A. Patterson. 2017. Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.

[35] David A. Patterson, Andrew Waterman. 2017. The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon.

[36] Benjamin C. Pierce. 2002. Types and Programming Languages. MIT Press.

[37] Xavier Leroy. 2009. Formal verification of a realistic compiler. Communications of the ACM 52(7):107–115.

[38] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP '23. arXiv:2309.06180.

[39] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI '22, pp. 521–538.

[40] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 35. arXiv:2205.14135.

[41] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. FAST '25, pp. 155–170. (Best Paper.)

[42] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, Vijay Gadepally. 2023. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. arXiv:2310.03003.

A Proof of Theorem 3.1 (Turing Completeness)

A.1 Strategy

We prove that AgentISA can simulate any program in the WHILE language over the natural numbers. WHILE programs compute exactly the partial recursive functions—WHILE is Turing equivalent [33, 36]—so a sound simulation of WHILE by AgentISA establishes Turing completeness. The argument is a counting argument over instruction classes: the instruction classes of Definition 3.1 must jointly provide unbounded storage, sequential composition, conditional branching, and unbounded iteration. We show the control-flow class $\mathcal{C}$ and the state-management class $\mathcal{S}$ suffice; the LLM-interaction class $\mathcal{L}$ is not needed, making the result conservative with respect to the full instruction set.

A.2 Source Language

Let WHILE be the language

$$P ::= x := x + 1 \mid x := x - 1 \mid x := y \mid \texttt{if } x = 0 \texttt{ then } P_1 \texttt{ else } P_2 \mid \texttt{while } x \neq 0 \texttt{ do } P \mid P_1; P_2$$

over variables ranging over $\mathbb{N}$ (with predecessor cut off at 0), with the standard small-step semantics. Configurations are pairs $\langle P, \sigma \rangle$ with store $\sigma : Var \to \mathbb{N}$.

A.3 Encoding

(i) Store. A WHILE store σ is represented by the AgentISA state $s_\sigma \in \Sigma$, in the component managed by the state-management instructions: `store` writes $\sigma(x)$ into the state cell for x, `load`

reads it, and `update` applies a unary arithmetic update. Σ is unbounded, which is the crucial ingredient of the counting argument: $\mathcal{S}$ provides an unbounded store.

(ii) Statements.

WHILE construct	AgentISA compilation
$x := y$	`load` y; `store` x
$x := x + 1$	`load` x; `update`$(+1)$; `store` x
$x := x - 1$	`load` x; `update`(-1); `store` x
`if` $x = 0$ `then` P_1 `else` P_2	`load` x; `branch`$(= 0?, \hat{P}_1, \hat{P}_2)$
`while` $x \neq 0$ `do` P	`loop`(`load` x; `test`$(\neq 0)$; $\hat{P}$)
$P_1; P_2$	$\hat{P}_1 \circ \hat{P}_2$ (sequential composition)

where $\hat{P}$ denotes the compilation of P. The no-op `skip` compiles to the identity instruction ϵ.

A.4 Simulation Lemma

Lemma A.1. For every WHILE configuration $\langle P, \sigma \rangle$ and its encoded AgentISA instruction sequence $\hat{P}$ started in state s_σ:

(a) if $\langle P, \sigma \rangle \to \langle P', \sigma' \rangle$, then $\hat{P}$ executed from s_σ reaches the encoding of $\langle P', \sigma' \rangle$ in finitely many AgentISA steps; and

(b) if $P =$ `skip` (terminal), then $\hat{P}$ terminates.

Proof. By structural induction on P:

- **Assignments** (base cases). Each compiled sequence performs exactly the corresponding store update; (a) holds by executing the two or three instructions in order, (b) trivially.
- **Conditional.** The `branch` instruction evaluates the zero test on the loaded value and transfers control to one of the compiled branch bodies. By the induction hypothesis applied to P_1 and P_2, each branch simulates its WHILE counterpart.
- **Loop.** The WHILE unfolding identity `while x $\neq$ 0 do P $\equiv$ if x = 0 then skip else (P ; while x $\neq$ 0 do P)` is simulated by the `loop` instruction's iteration: each iteration executes the guard test followed by $\hat{P}$. By induction on the number of executed iterations—the induction hypothesis applied to P covers each body execution, and the guard test discharges the loop condition—the k-th WHILE iteration is matched by the k-th `loop` iteration.
- **Sequence.** $\hat{P}_1 \circ \hat{P}_2$ executes $\hat{P}_1$ to completion, then $\hat{P}_2$; both simulations compose by the induction hypothesis.

The case arguments above are the substantive reduction steps; expanding each into a fully formal induction over the small-step rules (matching each WHILE step against the transition functions $\delta \in \Delta$ of the corresponding AgentISA instructions) is mechanical and follows the template of textbook simulation arguments [33, 36]. We mark those expansions as routine and leave them to the reader. ■

A.5 Conclusion

By Lemma A.1, every WHILE-computable function is AgentISA-computable. Since WHILE is Turing equivalent—a classical result of computability theory, with WHILE serving as the standard imperative model in semantics texts such as [36]—AgentISA is Turing complete. ■

A.6 Scope Remark

Turing completeness is a deliberately weak property—any language with unbounded storage and branching attains it. Its role here is negative certification: the instruction set does not artificially constrain what agent programs can express, so no expressiveness objection can be raised against the architecture. The claim says nothing about efficiency, and it is established without the LLM-interaction class $\mathcal{L}$, so it is conservative with respect to the full instruction set of Definition 3.1.

B Proof of Theorem 3.2 (Type Safety)

We prove type safety by the standard syntactic technique of simultaneous progress and preservation, in the style of Pierce [36].

B.1 Core Calculus

Restrict attention to the pure core of the instruction language: expressions over base types, λ-abstraction and application (whose β-reduction rule appears in Section 3.1.2), and the composition operators of Definition 3.6 applied to instructions carrying typed pre/post-conditions. The small-step relation $\rightarrow$ is generated by β-reduction, the context rule $E[e_1] \rightarrow E[e_1']$, and the per-instruction transition functions $\delta \in \Delta$. Effectful instructions are treated in B.7.

B.2 Lemma (Canonical Forms)

If $\vdash v : T$ and v is a value, then:

- $T \in \{Bool, Int, String, Float\}$ implies v is a literal of that type;
- $T = \Pi(x : T_1).T_2$ implies $v = \lambda x{:}T_1.\, e$ with $\Gamma, x{:}T_1 \vdash e : T_2$;
- $T = \Sigma(x : T_1).T_2$ implies $v = (v_1, v_2)$ with the components typed at the corresponding dependent types;
- $T = \mu X.T$ implies v is a rolled recursive value.

Proof. By inspection of the typing rules: each type has exactly one value form, and the syntax of values is defined per type, so the case analysis is exhaustive. This is the standard canonical-forms lemma [36]. ■

B.3 Lemma (Substitution)

If $\Gamma, x{:}T' \vdash e : T$ and $\vdash v : T'$, then $\Gamma \vdash e[x := v] : T$.

Proof. By induction on the derivation of $\Gamma, x{:}T' \vdash e : T$. All cases are standard; the only subtlety is variable capture in the Π-rule, handled by the usual α-conversion convention. The case analysis is that of Pierce [36] and we do not duplicate it. ■

B.4 Lemma (Progress)

If $\vdash e : T$, then e is a value or there exists e' with $e \to e'$.

Proof. By induction on the typing derivation:

- **Literals and** λ are values.
- **Application** $e_1\, e_2$ with $e_1 : \Pi(x : T_1).T_2$ and $e_2 : T_1$: by the induction hypothesis, either e_1 steps (then the context rule applies) or e_1 is a value, in which case B.2 gives $e_1 = \lambda x.\, e$; similarly e_2 either steps or is a value; when both are values, β-reduction applies. There is no stuck case, because the Π-type of e_1 and B.2 jointly guarantee a λ in value position.
- **Instruction composition** ($i_1 \circ i_2$, $i_1 \parallel i_2$, $i_1 \rhd i_2$, i_1^*): the typing rule for composition encodes Hoare-style sequencing—the postcondition of the first component must imply the precondition of the second. Hence, whenever execution reaches a composed instruction, the state satisfies the required precondition, and by Definition 2.2 the transition functions are total on precondition-satisfying states ($\forall i \in I, \exists \delta \in \Delta : \delta(s_t, i) \to s_{t+1}$). For iteration, either the guard is false (terminal) or the body steps by the induction hypothesis. ■

B.5 Lemma (Preservation)

If $\vdash e : T$ and $e \to e'$, then $\vdash e' : T$.

Proof. By induction on the typing derivation, with case analysis on the step:

- **β:** $(\lambda x.\, e)\, v \to e[x := v]$. Typing gives the body typed under $\Gamma, x{:}T_1$ and the argument at T_1; by B.3, the substituted term types at $T_2 = T$.
- **Context:** $E[e_1] \to E[e_1']$ with $e_1 \to e_1'$. By the induction hypothesis $e_1' : T_1$; an induction on the structure of E re-types $E[e_1']$ at the original type.
- **Instruction transitions:** the typing rule requires $post(i)$ to be a well-typed predicate of type T over the state; since i's transition function is certified to establish $post(i)$ (Definition 3.5, discharged per instruction by the bounded verification of Section 3.3.3), the reduct re-types at T.
- **Composition operators:** each transition rule is typed by the sequencing rule of B.4; the reduct re-types by the induction hypothesis on the component derivations.

The remaining instruction-class cases repeat this pattern with class-specific invariants; they are routine and omitted (the template is the standard preservation argument for the corresponding constructs [36]). ■

B.6 Theorem 3.2

By B.4, a well-typed closed term is a value or takes a step; by B.5, every step preserves typing. Hence no well-typed term reaches a stuck state: well-typed AgentISA programs cannot go wrong. ■

B.7 Scope Remarks

Two caveats state the theorem's boundaries honestly. First, B.4's appeal to totality of transition functions on precondition-satisfying states is an assumption about instruction *implementations*, discharged per instruction by the bounded verification of Section 3.3.3—not by the type system alone, and only as completely as that bounded check explores. Second, effectful instructions (LLM calls, resource allocation) can fail at runtime for reasons outside the type system (network errors,

exhausted capacity); Theorem 3.2 covers the pure core and effectful instructions with checkable preconditions. Runtime failure handling for the remaining cases is the responsibility of the fallback machinery of Section 5.6, not of types.

C Proofs of Theorems 3.4 and 3.5

C.1 Denotational Setup

Fix the state space Σ. For a pure instruction i, let $[\![i]\!] : \Sigma \to \Sigma$ denote its denotation, total on states satisfying $pre(i)$. Program denotation is defined by structural recursion on the composition algebra of Definition 3.6:

- $[\![i]\!]$ for atomic instructions (given);
- $[\![p_1 \circ p_2]\!] = [\![p_2]\!] \circ [\![p_1]\!]$ (note the order: p_1 executes first);
- $[\![p_1 \parallel p_2]\!](\sigma) = [\![p_2]\!]([\![p_1]\!](\sigma))$ for pure p_1, p_2—the order is immaterial because pure updates commute on their domains;
- $[\![p_1 \rhd p_2]\!]$ applies p_2 conditionally on p_1's outcome;
- $[\![p^*]\!]$ is the least fixed point of the corresponding functional.

Write $p \equiv q$ iff $[\![p]\!] = [\![q]\!]$ on all states where both are defined.

C.2 Lemma C.1 (Atomic Rewrite Soundness)

Each rewrite rule implemented by the optimizer (Section 3.4.4) preserves denotation.

(i) Fusion. $i_1 \circ i_2 \to i_{12}$, where the compiler constructs i_{12} with $pre(i_{12}) = pre(i_1)$, $post(i_{12}) = post(i_2)$, and a body implementing $[\![i_2]\!] \circ [\![i_1]\!]$ (intermediate bindings are named, so the composition is explicit). By construction $[\![i_{12}]\!] = [\![i_2]\!] \circ [\![i_1]\!] = [\![i_1 \circ i_2]\!]$. The compiler applies fusion only when output types of i_1 match input types of i_2 and the pair is pure or order-insensitive, since denotational equality of the fused form presupposes order-insensitivity of intermediate effects.

(ii) Dead-code elimination. Instruction i is removed from p when $Def(i) \cap Live_{out}(i) = \emptyset$, with liveness the standard fixpoint of $Live(i) = Use(i) \cup (Live_{out}(i) - Def(i))$ [33]. Claim: if i is dead and side-effect-free, then $p \equiv p \setminus \{i\}$. Proof sketch: by induction on the suffix program following i, its denotation depends only on the live variables at i's position; i writes none of them, so removing i cannot change any value subsequently read. The side-effect-freeness check is the same purity restriction as Definition 3.6's distributivity caveat—instructions declared `side_effects=True` are never eliminated.

(iii) Peephole combining. Each rule $(s_1, \ldots, s_k) \to s'$ carries the verification obligation $\forall \sigma : [\![s_1 \circ \ldots \circ s_k]\!](\sigma) = [\![s']\!](\sigma)$, checked at rule-definition time (Section 4.1.3). Instances: `LOAD; STORE` to the same location composes to `MOVE` (the value read is the value written); consecutive `QUERY` instructions with identical (instruction, params) pairs collapse to one cached query (this is the fusion instance underlying the caching measured in Section 5.2); consecutive `CONTEXT_UPDATE`s compose to a single update by associativity of the update function.

Proof. Each case reduces to the construction of the rewritten instruction and the definition of liveness. The inner inductions—on the pair of bodies for (i), on suffix programs for (ii)—are routine and left to the reader. ■

C.3 Lemma C.2 (Compositionality)

Let T be a program transform with $[\![T(p)]\!] = [\![p]\!]$ for all p. Then for any context $C[\cdot]$ built from the composition operators, $[\![C[T(p)]]\!] = [\![C[p]]\!]$.

Proof. By structural induction on C. Each operator's denotation (C.1) is defined compositionally in its constituents' denotations, so substituting a denotationally equal constituent leaves the whole unchanged. The parallel case uses purity (commutativity of pure updates). The iteration case uses the defining property of least fixed points: if the functionals induced by p and $T(p)$ are equal, then all finite approximants are equal (by the induction hypothesis), so their suprema—the least fixed points—are equal. ■

C.4 Theorem 3.4 (Optimization Soundness)

Proof. The compiler applies a finite sequence of rewrites $O = R_k \circ ... \circ R_1$ (finiteness follows from compilation terminating; each pass is a bounded traversal). By induction on k: $k = 0$ is trivial. For the step, write $O = R \circ O'$. By the induction hypothesis $[\![O'(p)]\!] = [\![p]\!]$. A single application of rewrite rule R preserves the denotation of the subprogram it rewrites (Lemma C.1), and Lemma C.2 lifts the preservation through any surrounding composition context. Hence $[\![R(O'(p))]\!] = [\![O'(p)]\!] = [\![p]\!]$. ■

C.5 Theorem 3.5 (Pipeline Correctness)

Let s be a valid instruction sequence and S a schedule satisfying:

- **(S1)** every instruction is dispatched exactly once;
- **(S2)** for every dependency $(i_1, i_2) \in \mathit{Deps}$: $\mathit{start}(i_2) \geq \mathit{end}(i_1)$ (the constraint of Section 3.4.3);
- **(S3)** instructions sharing a parallel dispatch group are pairwise non-interfering (no data hazard, no resource conflict);
- **(S4)** commit is in program order.

Then executing s under S yields the same final state as sequential execution: $[\![s]\!]_S = [\![s]\!]_{seq}$.

Proof. Let $g_1, ..., g_m$ be the dispatch groups of S in commit order. We show the invariant

$$I(t) : \text{after committing } g_1, ..., g_t,$$
$$\text{the architectural state equals the sequential state after}$$
$$\text{executing all instructions in those groups.}$$

- **Base** ($t = 0$): both sides are the initial state.
- **Step**: assume $I(t)$. Every instruction in g_{t+1} has its dependencies in earlier groups (by S2, since it is scheduled after them), which by $I(t)$ have completed and committed. Within the group, S3 gives pairwise non-interference: no instruction reads another group-member's output, and no two write the same location or resource. Hence the group's updates commute—for pure updates this is the diamond property, and for effectful instructions S3's resource-conflict exclusion forces serialization at the resource, again yielding a unique combined effect. The state after g_{t+1} is therefore independent of the group's internal interleaving and equals the sequential execution of the same instructions from the state at $I(t)$.
- By induction, $I(m)$ holds; by S1 no instruction was skipped or duplicated. ■

Corollary (dispatch folding). The ADVANCED and AGGRESSIVE levels fold independent instructions into shared dispatches (Section 5.3). A folded group satisfies S3 by construction, so by Theorem 3.5 the fold changes the *dispatch count* but neither the semantics nor—consistent with the measurement of Section 5.3—the effective instruction count.

Theorem 3.6 follows by the same machinery: every optimization in the pass pipeline is a finite composition of rewrites covered by Lemma C.1, and the argument of C.4 applies verbatim. ■

C.6 Scope Remark

The proofs assume the verification obligations of Sections 4.1.3 and 4.3.1 hold: peephole rules verified at definition time, and transformation equivalence checked by symbolic execution. Where an obligation is discharged only on bounded state spaces (Section 3.3.3), the guarantee is correspondingly bounded—rewrites verified on the explored subset are sound there, and the compiler restricts their application accordingly.